Building a SQL Data Warehouse: A Step-by-Step Journey

David Omar · Follow · 2 min read · Apr 15, 2025

Published on 15 Apr 2025

Get David Omar's stories in your inbox
Join Medium for free to get updates from this writer.
Enter your email · Subscribe

In this post, I'll walk you through the development of my SQL Data Warehouse project. I'll explain the key steps, share the code snippets, and highlight the challenges and solutions along the way. This project leverages a multi-layered architecture (Bronze, Silver, and Gold) to ensure data quality and deliver business-ready analytics.

Project Overview

The main objective of this project was to design and implement a data warehouse that ingests raw data, transforms it into cleansed data, and finally produces a curated layer for analytical consumption. The layered approach is as follows:

- **Bronze Layer**: Raw data ingestion and initial staging.
- **Silver Layer**: Data transformation, cleansing, and enrichment.
- **Gold Layer**: Creation of dimensional views and fact tables for business analytics.

SQL Data Warehouse Architecture

Step 1: Database & Schema Setup

The journey begins with creating a new database and defining schemas for each layer. Before running the scripts, ensure you have backups since the process drops and recreates the database if it exists.

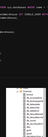
```
USE Master;
GO

IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'DataWarehouse')
BEGIN
    ALTER DATABASE DataWarehouse SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE DataWarehouse;
END;
GO

CREATE DATABASE DataWarehouse;
GO

USE DataWarehouse;

GO
CREATE SCHEMA bronze;
GO
CREATE SCHEMA silver;
GO
CREATE SCHEMA gold;
GO
```



Schemas

Step 2: Creating the Bronze Layer

The Bronze layer is where the raw data lands. I created tables for customers, product, sales, and ERP data. The scripts include drop and create commands to ensure a fresh start every time.

```
IF OBJECT_ID('bronze.crm_cust_info', 'U') IS NOT NULL
    DROP TABLE bronze.crm_cust_info;
GO

CREATE TABLE bronze.crm_cust_info (
    cst_id              INT,
    cst_key             NVARCHAR(50),
    cst_firstname       NVARCHAR(50),
    cst_lastname        NVARCHAR(50),
    cst_marital_status  NVARCHAR(50),
    cst_gndr            NVARCHAR(50),
    cst_create_date     DATE
);
GO
```

Initial tables

Step 3: Loading Data Into the Bronze Layer

A stored procedure was created to load CSV data into the Bronze tables. This procedure:

- Truncates existing data.
- Uses `BULK INSERT` to load data from external CSV files.
- Logs the load duration for performance monitoring.

Example: Bronze Data Loading Stored Procedure

```
ALTER PROCEDURE [bronze].[load_bronze] AS
BEGIN
    DECLARE @start_time DATETIME, @end_time DATETIME;
    BEGIN TRY
        PRINT '>> Truncating Table: bronze.crm_cust_info';
        TRUNCATE TABLE bronze.crm_cust_info;
        PRINT '>> Inserting Data Into: bronze.crm_cust_info';
        BULK INSERT bronze.crm_cust_info
        FROM 'C:\sql-data-warehouse-project\datasets\source_crm\cust_info.csv'
        WITH (
            FIRSTROW = 2,
            FIELDTERMINATOR = ',',
            TABLOCK
        );
        SET @end_time = GETDATE();
        PRINT '>> Load Duration: ' + CAST(DATEDIFF(second, @start_time, @end_time)
AS NVARCHAR) + ' seconds';
        PRINT '>> -------------';
    END TRY
    BEGIN CATCH
        PRINT '>> Error during Bronze Layer Load! - ' + ERROR_MESSAGE();
    END CATCH
END
GO
```

Data flow to Bronze layer

Step 4: Transforming Data in the Silver Layer

The Silver layer transforms raw data into a standardized format. I created new tables and stored procedures that:

- Cleanse the data (e.g., trimming spaces, normalizing gender and marital status).
- Transform date formats and calculate derived columns (like end dates for product lifecycles).

Example: Silver Layer Transformation

```
CREATE TABLE silver.crm_cust_info (
    cst_id              INT,
    cst_key             NVARCHAR(50),
    cst_firstname       NVARCHAR(50),
    cst_lastname        NVARCHAR(50),
    cst_marital_status  NVARCHAR(50),
    cst_gndr            NVARCHAR(50),
    cst_create_date     DATE,
    dwh_create_date     DATETIME2 DEFAULT GETDATE()
);
GO
```

And a stored procedure to load transformed data:

```
ALTER OR ALTER PROCEDURE [silver].[load_silver] AS
BEGIN
    -- Insert cleaned data into silver.crm_cust_info from Bronze to Silver
    INSERT INTO silver.crm_cust_info (
        cst_id, cst_key, cst_firstname, cst_lastname, cst_marital_status, cst_gndr,
        cst_create_date
    )
    SELECT
        cst_id,
        cst_key,
        TRIM(cst_firstname),
        TRIM(cst_lastname),
        CASE WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
             WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
             ELSE 'n/a'
        END,
        CASE WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
             WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
             ELSE 'n/a'
        END,
        cst_create_date
    FROM bronze.crm_cust_info;
GO
```

Before and After transformation in Silver layer (Bronze data layer)

Step 5: Building Business-Ready Gold Views

The Gold layer provides a clean, analytical interface over the transformed data. I created views to join fifteen data from dimensional models and fact tables.

Example: Creating a Gold View for Customers

```
CREATE VIEW gold.dim_customers AS
SELECT
    ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,
    ci.cst_id                           AS customer_id,
    ci.cst_key                          AS customer_number,
    ci.cst_firstname                    AS first_name,
    ci.cst_lastname                     AS last_name,
    la.cntry                            AS country,
    ci.cst_marital_status               AS marital_status,
    CASE WHEN ci.cst_gndr <> 'n/a' THEN ci.cst_gndr
         ELSE COALESCE(ca.gen, 'n/a')
    END                                 AS gender,
    ca.bdate                            AS birthdate,
    ci.cst_create_date                  AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca
    ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
    ON ci.cst_key = la.cid
ORDER BY ci.cst_id;
GO
```

Views

gold.dim_customers view

Gold layer Star schema

Step 6: Quality Checks and Data Validation

To ensure data integrity, I implemented several quality checks across the Silver and Gold layers. These checks include:

- Duplicate or null primary keys.
- Unwanted spaces in string fields.
- Data consistency between related fields.
- Validity of date fields and relationships.

Example: Quality Check for CRM Customer Info

```
-- Check for NULLs or duplicates in primary key
SELECT
    cst_id,
    COUNT(*)
FROM silver.crm_cust_info
GROUP BY cst_id
HAVING COUNT(*) > 1 OR cst_id IS NULL;
```

Efficient data quality checks for each layer (bronze

Final Thoughts

This SQL data warehouse project has been a rewarding experience. By implementing a layered architecture, I was able to:

- Efficiently ingest and transform raw data.
- Achieve data quality through structured checks.
- Produce business-ready views for insightful analytics.

The approach not only enhanced data consistency but also set a solid foundation for scaling analytics in the future.

Find the source code and the full project on GitHub for complete scripts and further details.

Sql · Data Warehouse · Sql Server · Data Engineering · 4M

Written by David Omar
0 Followers · 1 Following
Data Analyst & Business Intelligence Specialist | Customer Service & Process Management
Connect with me on LinkedIn: www.linkedin.com/in/davidomar-omar

Follow

No responses yet
Write a response
What are your thoughts?

More from David Omar

David Omar · Apr 15, 2025
Analyzing NY 311 Data: A Cross-Into Power and Team Statistics with...
Introduction: The Power of Data Analytics in Modern Government Services Living in New...
Apr 15, 2025 · 1

David Omar · Oct 29, 2024
Comprehensive Analysis of COVID-19 Data Using SQL
Introduction
Oct 29, 2024

David Omar · Oct 27, 2024
Automating Job Vacancy Sourcing using Nifty Scrapping with Python...
Introduction: Automating Job Vacancy Sourcing with Nifty Scrapping and Python...
Oct 27, 2024 · 1

David Omar · Oct 27, 2024
Using Python to Identify Popular App Genres and Maximize...
Introduction: Leveraging Data Analytics for Strategic Decision-Making In the...
Oct 27, 2024

See all from David Omar

Recommended from Medium

Kanan Gulali · Apr 15, 2025
MySQL Stored Procedure Optimization: Transforming Slow...
Introduction: Why Stored Procedures Matter in Big Data Environments
Apr 15, 2025 · 1

Kamala Mishra · Mar 31, 2025
Why I Stopped Using CTEs Everywhere in SQL
Sometimes less is more. Learn when to use CTEs and when to avoid them for speed, clarity and multiple uses ...
Mar 31, 2025 · 1

Vaibhav Gupta · Jan 2, 2026
Complete Data Engineer Roadmap with 3000+ Free Learning
Complete Data Engineer Roadmap with 3000+ Free Learning Resources Introduction: Why Data Engineering Matters ...
Jan 2, 2026 · 1

CMC - The Ritual Lab · Aug 25, 2025
MERGE in SQL
The UPSERT You Should Be Using
Aug 25, 2025

Kartik Kapoor · Jan 27, 2026
Top 50 Snowflake SQL Functions Every Data Professional Should...
Snowflake SQL functions are the building blocks of data transformation. Whether you're cleaning messy data, ...
Jan 27, 2026 · 1

Abhinav Sharma · Jan 27, 2026
Modern Data Architectures: Relational, Layered, and Lakehouse
Data architecture defines how data is stored, organized, and made available for analysis. Different architectural ...
Jan 27, 2026 · 1

See more recommendations

Help · Status · About · Careers · Press · Blog · Privacy · Rules · Terms · Text to speech